

```
1
2 // COS30008, List, Problem Set 3, 2022
3
4 #pragma once
5
6 #include "DoublyLinkedList.h"
7 #include "DoublyLinkedListIterator.h"
8
9 #include <iostream>
10 #include <stdexcept>
11 #include <utility>
12
13 template <typename T> class List {
14     private:
15         // auxiliary definition to simplify node usage
16         using Node = DoublyLinkedList<T>;
17
18         Node *fRoot; // the first element in the list
19         size_t fCount; // number of elements in the list
20
21     public:
22         // auxiliary definition to simplify iterator usage
23         using Iterator = DoublyLinkedListIterator<T>;
24
25         ~List() // destructor - frees all nodes
26         {
27             while (fRoot != nullptr) {
28                 if (fRoot != &fRoot->getPrevious()) // more than one element
29                 {
30                     Node *lTemp =
31                         const_cast<Node *>(&fRoot->getPrevious()); // select
32                             last
33
34                     lTemp->isolate(); // remove from list
35                     delete lTemp; // free
36                 } else {
37                     delete fRoot; // free last
38                     break; // stop loop
39                 }
40             }
41
42         void remove(const T &aElement) // remove first match from list
43         {
44             Node *lNode = fRoot; // start at first
45
46             while (lNode != nullptr) // Are there still nodes available?
47             {
48                 if (**lNode == aElement) // Have we found the node?
```

```

49         {
50             break; // stop the search
51         }
52
53         if (lNode != &fRoot->getPrevious()) // not reached last
54         {
55             lNode = const_cast<Node *>(&lNode->getNext()); // go to next
56         } else {
57             lNode = nullptr; // stop search
58         }
59     }
60
61     // At this point we have either reached the end or found the node.
62     if (lNode != nullptr) // We have found the node.
63     {
64         if (fCount != 1) // not the last element
65         {
66             if (lNode == fRoot) {
67                 fRoot =
68                     const_cast<Node *>(&fRoot->getNext()); // make next root
69             }
70         } else {
71             fRoot = nullptr; // list becomes empty
72         }
73
74         lNode->isolate(); // isolate node
75         delete lNode;    // release node's memory
76         fCount--;        // decrement count
77     }
78 }
79
80 ///////////////////////////////////////////////////
81 /// PS3
82 ///////////////////////////////////////////////////
83
84 // P1
85
86 // default constructor
87 List() {
88     this->fRoot = nullptr;
89     this->fCount = 0;
90 }
91
92 // Is list empty?
93 bool empty() const { return this->fCount == 0 || this->fRoot ==
94     nullptr; }

```

```
95 // list size
96 size_t size() const { return this->fCount; };
97
98 // adds aElement at front
99 void push_front(const T &aElement) {
100     Node *node = new Node(aElement);
101
102     if (this->fRoot == nullptr) {
103         this->fRoot = node;
104     } else {
105         this->fRoot = &this->fRoot->push_front(*node);
106         // this->fRoot = node;
107     }
108
109     this->fCount += 1;
110 }
111
112 // return a forward iterator
113 Iterator begin() const {
114     return DoublyLinkedListIterator<T>(this->fRoot).begin();
115 }
116
117 // return a forward end iterator
118 Iterator end() const {
119     return DoublyLinkedListIterator<T>(this->fRoot).end();
120 }
121
122 // return a backwards iterator
123 Iterator rbegin() const {
124     return DoublyLinkedListIterator<T>(this->fRoot).rbegin();
125 }
126
127 // return a backwards end iterator
128 Iterator rend() const {
129     return DoublyLinkedListIterator<T>(this->fRoot).rend();
130 }
131
132 // P2
133 // adds aElement at back
134 void push_back(const T &aElement) {
135     Node *node = new Node(aElement);
136
137     if (this->fRoot == nullptr) {
138         this->fRoot = node;
139     } else {
140         this->fRoot->push_front(*node);
141     }
142
143     this->fCount += 1;
```

```
144     }
145
146     // P3
147     // list indexer
148     const T &operator[](size_t aIndex) const {
149         if (aIndex >= this->fCount) {
150             throw std::out_of_range("Index out of bounds");
151         }
152         auto it = this->begin();
153         for (size_t i = 0; i < aIndex; i++) {
154             ++it;
155         }
156         return *it;
157     }
158
159     // P4
160
161     // copy constructor
162     List(const List &aOtherList) {
163         // perform a deep copy of aOtherList, since assignment of
164         // aOtherList.fRoot basically points to the same pointer
165         this->fRoot = nullptr;
166         this->fCount = 0;
167
168         for (int i = 0; i < aOtherList.fCount; i++) {
169             this->push_back(aOtherList[i]);
170         }
171     }
172
173     // assignment operator
174     List &operator=(const List &aOtherList) {
175         this->fRoot = nullptr;
176         this->fCount = 0;
177
178         for (int i = 0; i < aOtherList.fCount; i++) {
179             this->push_back(aOtherList[i]);
180         }
181         return *this;
182     }
183
184     // P5
185
186     // move constructor
187     List(List &&aOtherList) noexcept {
188         this->fRoot = std::exchange(aOtherList.fRoot, nullptr);
189         this->fCount = std::exchange(aOtherList.fCount, 0);
190     }
191
192     // move assignment operator
```

```
193     List &operator=(List &&aOtherList) noexcept {
194         // test
195         this->fRoot = std::exchange(aOtherList.fRoot, nullptr);
196         this->fCount = std::exchange(aOtherList.fCount, 0);
197
198         return *this;
199     }
200
201     // move push_front
202     // operate on lvalues, so we need to change them into rvalues first
203     void push_front(T &&aElement) {
204         Node *node = new Node(std::move(aElement));
205
206         if (this->empty()) {
207             this->fRoot = node;
208         } else {
209             this->fRoot = &this->fRoot->push_front(*node);
210             // this->fRoot = node;
211         }
212
213         this->fCount += 1;
214     }
215
216     // move push_back
217     // same idea
218     void push_back(T &&aElement) {
219         Node *node = new Node(std::move(aElement));
220
221         if (this->fRoot == nullptr) {
222             this->fRoot = node;
223         } else {
224             this->fRoot->push_front(*node);
225         }
226
227         this->fCount += 1;
228     }
229 };
230
```